

Pandas for Data Analysis

Loading, cleaning, transforming and summarizing data in Python

In this guide

- Introduction
- 1. Series and DataFrames: The Building Blocks
- 2. Loading and Inspecting Data
- 3. Selecting, Filtering and Sorting
- 4. Cleaning Data
- 5. Transforming, Grouping and Aggregating
- 6. Combining, Visualizing and Exporting
- Conclusion

Introduction

Pandas is the most widely used Python library for working with tabular data. It provides two core objects, the Series (a labeled column) and the DataFrame (a table of columns), along with fast, expressive tools for reading files, cleaning messy values, reshaping data and computing summaries. If you have used spreadsheets or SQL, Pandas will feel familiar, but it adds the full power of Python.

This guide follows a typical analysis: load a dataset, inspect it, select and filter, clean it, transform and group it, combine multiple sources, and finally visualize and export results. All examples assume the following imports:

```
import pandas as pd
import numpy as np
```

Why Pandas?

- **Productive:** operations that take many lines in plain Python take one.
- **Fast:** built on NumPy arrays and optimized C code.
- **Flexible:** reads CSV, Excel, JSON, SQL databases, Parquet and more.
- **Integrated:** works with Matplotlib, scikit-learn and the wider data ecosystem.

A running example

Most snippets use a small sales dataset with columns **date**, **region**, **product**, **units** and **price**. Imagine it is stored in **sales.csv**.

date	region	product	units	price
2025-01-03	North	Widget	10	2.50
2025-01-04	South	Gadget	4	9.99
2025-01-04	North	Gadget	7	9.99

Install: Run `pip install pandas` (Jupyter notebooks are a very convenient place to experiment).

1. Series and DataFrames: The Building Blocks

A **Series** is a one-dimensional array with an index, and a **DataFrame** is a two-dimensional table whose columns are Series sharing the same index. You can create them from lists, dictionaries or files.

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s["b"])                                # 20

df = pd.DataFrame({
    "region": ["North", "South", "North"],
    "units":  [10, 4, 7],
    "price":  [2.50, 9.99, 9.99],
})
df["revenue"] = df["units"] * df["price"]    # vectorized: no loop needed
print(df)
```

Vectorized operations

Operations on whole columns are applied to every element at once. This is both simpler and dramatically faster than looping over rows. Prefer `df["a"] + df["b"]` over a `for` loop.

Data types (dtypes)

Each column has a single dtype, which affects memory and available operations. Check with `df.dtypes` and convert with `astype`.

dtype	Holds	Convert with
<code>int64, float64</code>	Numbers	<code>astype("float64")</code>
<code>object / string</code>	Text	<code>astype("string")</code>
<code>bool</code>	True / False	<code>astype(bool)</code>
<code>datetime64[ns]</code>	Dates and times	<code>pd.to_datetime(col)</code>
<code>category</code>	Repeated labels	<code>astype("category")</code>

Key points

- A DataFrame is a dictionary-like collection of Series.
- Use vectorized column arithmetic instead of loops.
- Correct dtypes save memory and unlock date and category features.

2. Loading and Inspecting Data

Pandas can read many formats with a consistent family of `read_*` functions. CSV is the most common.

```
df = pd.read_csv("sales.csv", parse_dates=["date"])

df_x = pd.read_excel("report.xlsx", sheet_name="Q1")
df_j = pd.read_json("data.json")
# df_sql = pd.read_sql("SELECT * FROM orders", connection)
```

- `sep=";"` handles other delimiters; `usecols=[...]` loads only the columns you need.
- `dtype={...}` forces types; `na_values=["-", "n/a"]` marks custom missing markers.
- `nrows=1000` is useful for peeking at very large files.

First look: the inspection toolkit

Method	What it shows
<code>df.head()</code> / <code>df.tail()</code>	First or last rows
<code>df.shape</code>	(rows, columns)
<code>df.info()</code>	Column names, dtypes, non-null counts, memory
<code>df.describe()</code>	Count, mean, std, min, quartiles, max for numeric columns
<code>df.isna().sum()</code>	Missing values per column
<code>df["region"].value_counts()</code>	Frequency of each category
<code>df.nunique()</code>	Number of unique values per column

```
print(df.shape)                # (1200, 5)
df.info()
print(df.describe())
print(df["region"].value_counts(normalize=True))
```

Saving results

```
df.to_csv("clean_sales.csv", index=False)
df.to_parquet("clean_sales.parquet")    # compact and fast for big data
```

Key points

- Always run `head`, `info` and `describe` before analyzing.
- Check missing values and dtypes immediately.
- Use `index=False` when saving unless the index is meaningful.

3. Selecting, Filtering and Sorting

Choosing the right rows and columns is the daily work of data analysis. Pandas offers several selection tools, and understanding when to use each one prevents many bugs.

Selecting columns

```
df["units"]          # one column -> Series
df[["region", "units"]] # several columns -> DataFrame
```

loc and iloc

- **loc** selects by **label** (row index labels and column names); slices include the end.
- **iloc** selects by **integer position**; slices exclude the end, like Python lists.

```
df.loc[0, "region"]          # row label 0, column "region"
df.loc[2:4, ["units", "price"]] # rows labeled 2..4 inclusive
df.iloc[0:3, 0:2]            # first 3 rows, first 2 columns
```

Boolean filtering

Create a True/False mask by comparing a column with a value, then use it to keep matching rows. Combine conditions with **&** (and), **|** (or) and **~** (not), wrapping each condition in parentheses.

```
north = df[df["region"] == "North"]
big_north = df[(df["region"] == "North") & (df["units"] >= 10)]
picked = df[df["product"].isin(["Widget", "Gadget"])]
q = df.query("units > 5 and price < 5")    # readable alternative
```

Common error: Using Python's **and/or** instead of **&|**, or forgetting parentheses, raises a ValueError.

Sorting

```
df.sort_values("revenue", ascending=False).head(10)
df.sort_values(["region", "units"], ascending=[True, False])
df.nlargest(5, "revenue")
```

Key points

- **loc** uses labels, **iloc** uses positions.
- Wrap each boolean condition in parentheses.
- Selections may return views or copies; use **.copy()** before modifying a subset.

4. Cleaning Data

Real datasets contain missing values, duplicates, inconsistent labels and wrong types. Analysts commonly spend most of their time on cleaning, and Pandas provides direct tools for each problem.

Missing values

```
df.isna().sum()                # count missing per column
df = df.dropna(subset=["price"]) # drop rows missing price
df["units"] = df["units"].fillna(0) # fill with a constant
df["price"] = df["price"].fillna(df["price"].median()) # fill with statistic
```

Choose a strategy deliberately: dropping loses information, while filling can distort distributions. Sometimes the fact that a value is missing is itself informative.

Duplicates

```
print(df.duplicated().sum())
df = df.drop_duplicates()
df = df.drop_duplicates(subset=["date", "region", "product"], keep="last")
```

Fixing types and text

```
df["date"] = pd.to_datetime(df["date"], errors="coerce")
df["units"] = pd.to_numeric(df["units"], errors="coerce")
df["region"] = df["region"].str.strip().str.title()
df["product"] = df["product"].str.lower().replace({"widget": "widgit"})
```

`errors="coerce"` turns unparseable values into NaT/NaN instead of raising an error, so you can find and handle them afterwards. The `.str` accessor exposes text methods such as `strip`, `lower`, `replace`, `contains` and `split`.

Outliers

Extreme values may be errors or genuine events. Inspect them with plots and summary statistics before removing anything. A common rule flags values beyond 1.5 times the interquartile range (IQR) from the quartiles.

```
q1, q3 = df["units"].quantile([0.25, 0.75])
iqr = q3 - q1
mask = df["units"].between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
clean = df[mask]
```

Renaming and dropping

```
df = df.rename(columns={"units": "units_sold"})
df = df.drop(columns=["internal_note"])
```

5. Transforming, Grouping and Aggregating

After cleaning, you usually create new columns and summarize the data. The **split-apply-combine** pattern, implemented by **groupby**, is the workhorse: split rows into groups, apply a calculation to each group and combine the results.

Creating and transforming columns

```
df["revenue"] = df["units"] * df["price"]
df["month"] = df["date"].dt.to_period("M")
df["size"] = np.where(df["units"] >= 10, "bulk", "single")
df["price_band"] = pd.cut(df["price"], [0, 5, 10, 100],
                          labels=["low", "mid", "high"])
df["revenue_clean"] = df["revenue"].apply(lambda v: round(v, 1))
```

groupby

```
df.groupby("region")["revenue"].sum()

summary = (df.groupby(["region", "product"])
            .agg(total_units=("units", "sum"),
                 avg_price=("price", "mean"),
                 orders=("date", "count"))
            .reset_index()
            .sort_values("total_units", ascending=False))
print(summary)
```

Named aggregation, shown above, produces readable column names. `reset_index()` turns the group keys back into ordinary columns.

Pivot tables and crosstabs

```
pt = df.pivot_table(index="region", columns="product",
                     values="revenue", aggfunc="sum", fill_value=0)

pd.crosstab(df["region"], df["size"])    # counts of combinations
```

Time-based analysis

```
daily = df.set_index("date").resample("W")["revenue"].sum()    # weekly totals
df["running_total"] = df["revenue"].cumsum()
df["rolling_avg"] = df["revenue"].rolling(7).mean()
```

Key points

- Use **groupby** plus **agg** for summaries by category.
- **pivot_table** reshapes summaries into a cross-tab layout.
- The **.dt** accessor unlocks year, month, weekday and other date parts.

6. Combining, Visualizing and Exporting

Analysis usually involves more than one table, and results must eventually be shared. This final section covers merging datasets, quick plotting and performance habits.

Merging (joining) tables

`merge` works like a SQL join, matching rows on one or more key columns. The `how` argument selects the join type: `inner`, `left`, `right` or `outer`.

```
stores = pd.DataFrame({"region": ["North", "South"],
                        "manager": ["Ira", "Raj"]})

full = df.merge(stores, on="region", how="left")
print(full.head())
```

Stacking tables

```
all_months = pd.concat([jan_df, feb_df, mar_df], ignore_index=True) # stack rows
wide = pd.concat([df_a, df_b], axis=1) # side by side
```

Check your joins: After a merge, compare row counts before and after. A jump in rows usually means duplicate keys created a many-to-many match.

Quick visualization

Pandas integrates with Matplotlib, so any Series or DataFrame can be plotted with one call.

```
import matplotlib.pyplot as plt

df.groupby("region")["revenue"].sum().plot(kind="bar", title="Revenue by region")
df["units"].plot(kind="hist", bins=20)
plt.tight_layout()
plt.show()
```

Performance and good habits

- Avoid `iterrows()` and row loops; use vectorized operations, `groupby` or `apply` only when necessary.
- Convert repeated strings to `category` dtype to save memory.
- Read only needed columns and use Parquet for large data.
- Chain operations with parentheses and keep transformation steps in a readable pipeline.
- Never modify the raw file; keep cleaning steps in a script so results are reproducible.

A compact end-to-end pipeline

```
result = (pd.read_csv("sales.csv", parse_dates=["date"])
          .drop_duplicates()
          .dropna(subset=["units", "price"])
          .assign(revenue=lambda d: d["units"] * d["price"])
          .groupby("region", as_index=False)["revenue"].sum()
          .sort_values("revenue", ascending=False))
result.to_csv("revenue_by_region.csv", index=False)
```

Conclusion

Pandas turns raw files into insight with only a few lines of code. You have seen how Series and DataFrames represent data, how to load and inspect datasets, select and filter with `loc`, `iloc` and boolean masks, clean missing and duplicate values, summarize with `groupby` and pivot tables, and finally merge, plot and export your results.

Key takeaways

- Inspect first: `head`, `info`, `describe` and `isna().sum()`.
- Prefer vectorized operations to loops.
- Parenthesize boolean conditions and use `&`, `|`, `~`.
- Treat cleaning as a documented, repeatable step, not a manual edit.
- Verify row counts after every merge.

Suggested next steps

- 1 Analyze a public dataset (for example from Kaggle or data.gov) from load to chart.
- 2 Learn NumPy basics, plotting with Matplotlib and Seaborn, and time-series features.
- 3 Explore MultiIndex, window functions and `pd.melt` for reshaping wide data.
- 4 Try Polars or DuckDB for very large datasets and compare performance.

Remember: Good analysis is reproducible. Keep your code and notes together so anyone, including future you, can rerun the work.